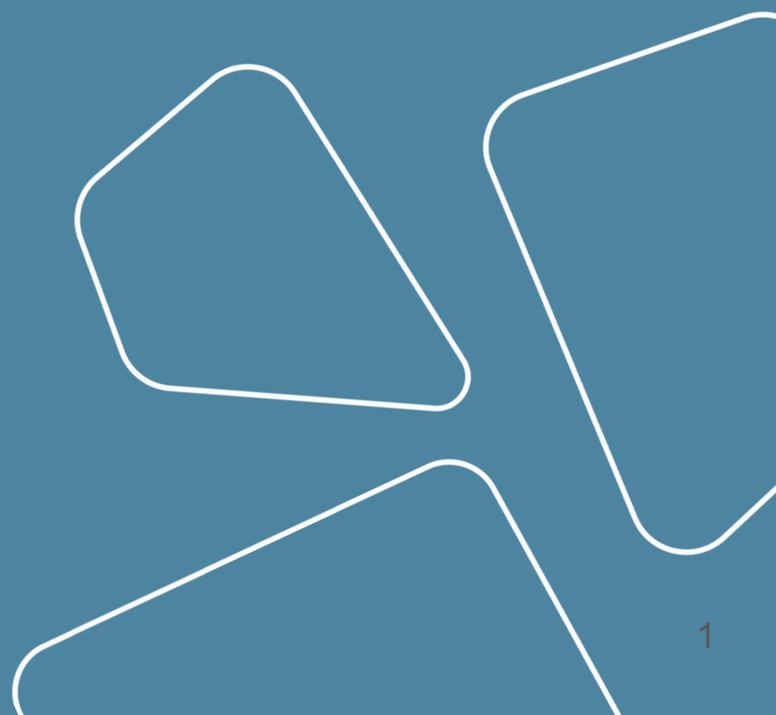
Computational resources

Vladislav Goncharenko

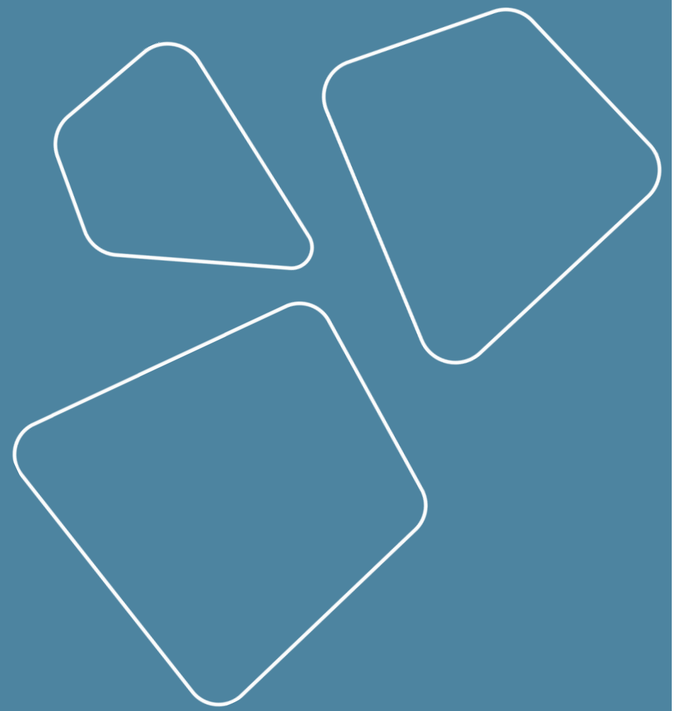
MIPT, fall 2026



girafe ai

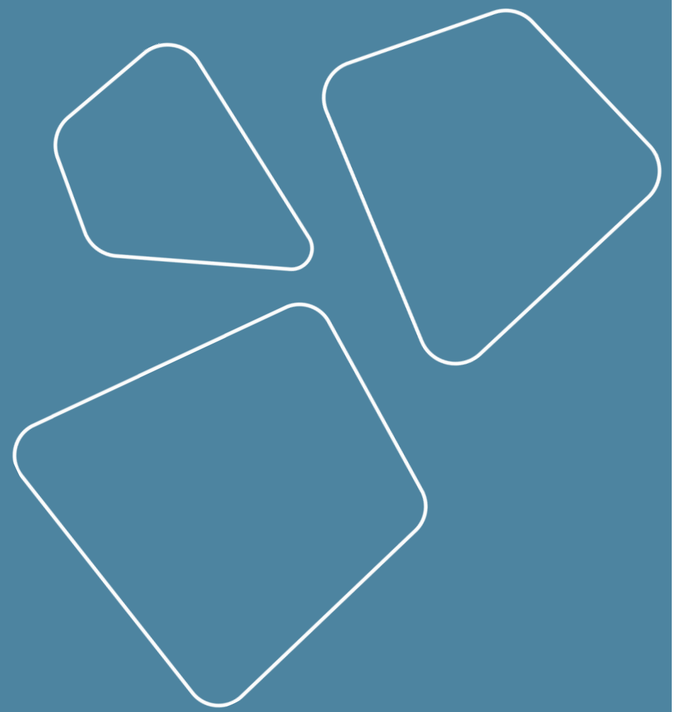


Recap



1. Logging
    - MLflow
2. Visualization
3. Frameworks for training
    - Lightning
    - Hugging face

Outline



1. Contours of computation
    a. Offline
    b. Online (realtime)
    c. Near real time
    d. Edge
2. Types of compute resources
3. Jobs runners

Note on server security

You have to set up security on your Internet facing server
e.g. **disable password authentication** because every minute you are scanned by various bots.

Many cloud providers require ssh key even before they allocate a VPS for you.

https://www.youtube.com/watch?v=DGeuS3-wN10

Motivation for computation management

- Scaling of computations
  - Typical clusters are 3-10k hardware servers
- Workload optimization
  - One engineer doesn’t need hardware fulltime

Split machines by function

- Development machine
  - medium resources
  - free use
  - persistent environment
  - Jupyter hub / per user allocated VMs
- Worker machine
  - high resources
  - managed use
  - per task environment
  - various management systems
- Production machine
  - ...

Contours of computation

girafe
ai

01

Offline

Operations:

- can take arbitrary time (from 100s of ms to 10s of days)
- can take arbitrary compute (from 1 core to 10k)
- can go down without business impact
- have a relatively low execution priority

Examples:

- Building daily analytics
- EDA (ELT)
- Model training
- Model inference (in some cases)

Online (aka realtime)

Real-time computing (RTC) is the computer science term for hardware and software systems subject to a "real-time constraint", for example from event to system response. Real-time programs must guarantee response within specified time constraints, often referred to as "deadlines".

Online (aka realtime)

Operations:

- must meet estimated time of arrival (ETA) (usually < 1 second)
- use predefined amount of resources
- failure causes apps to break
- have highest priority

Examples:

- site rendering
- quotes API of a stock market
- databases writes
- ML models inference
  - search engine results, recommender systems, advertisements
  - route generation on maps

Near real time

The term "near real-time" or "nearly real-time" (NRT), in telecommunications and computing, refers to the time delay introduced, by automated data processing or network transmission, between the occurrence of an event and the use of the processed data, such as for display or feedback and control purposes.

Near real time

Operations:

- must meet estimated time of arrival (ETA) (usually < 10 minutes)
- use predefined amount of resources
- failure needs to be fixed with a bit relaxed deadline
- have high priority

Examples:

- site monitoring rendering
- spam detection
- speech to text in instant messaging
- video processing
  - transcoding
  - video generation
- ALS models computing
- CI/CD processes
- chatbots (to some extent)
- embedding building for new items posted
- Food delivery/Taxi request posting

Edge

Operations:

- must meet estimated time of arrival (ETA) (usually < 10-100 ms)
- use predefined amount of resources (stored on device)
- failure causes apps to break
- have high priority

Examples:

- IoT
- CCTV analysis
- visual editors on smartphones
- bio-identification, biometry
- smart watch (num of steps, heart rate, state of mind)
- self-driving cars
- autocomplete (T9)

Main characteristics

- latency / deadline
  - offline
  - near-realtime
  - online
- caller coupling
  - synchronous
  - asynchronous
- data arrival model
  - batch
  - stream
- execution trigger: scheduled, event-driven, continuous

Compute speed metrics

girafe ai

02

Compute resource metrics

Main metrics are

- Latency
  - how much time we wait between putting task to queue and receiving final result
- Throughput
  - how many tasks do we accomplish on average in time unit

They may behave differently depending on load

- disk and compute utilization
- network in/out size
- request/response rate
- uptime

Cluster management systems

girafe ai

03

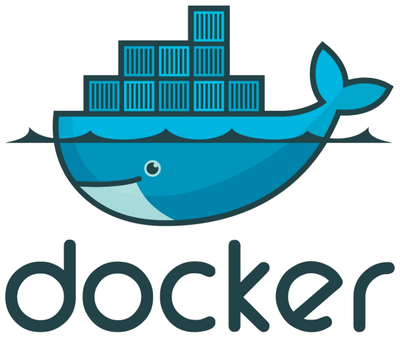
Cluster management models

- MapReduce
  - data processing
- Task queues
  - simple computations
- Orchestrators
  - complex computations and configurations
- Serverless computing
  - streaming computing managed by cloud

Examples

- MapReduce
  - Hadoop
  - YTsaurus
- Task queues
  - slurm
  - clearml
  - Regular job runners
    - cron
    - airflow
- Orchestrators
  - Docker Swarm
  - Kubernetes (k8s)
- Serverless
  - Amazon Lambda
  - Yandex Function







kubernetes



Cron string



Format of a cron string:

```
* * * * *  command_to_execute
```

Each of the five asterisks (*) corresponds to a specific time unit in the following order:

- **Minute (0-59)**: The minute of the hour when the command will run.
- **Hour (0-23)**: The hour of the day when the command will run.
- **Day of the month (1-31)**: The day of the month when the command will run.
- **Month (1-12 or jan-dec)**: The month of the year when the command will run.
- **Day of the week (0-6 or sun-sat)**: The day of the week when the command will run (0 = Sunday, 6 = Saturday).

https://www.baeldung.com/cron-expressions

Regular job runs

The de facto industry standard is Airflow

Other stacks (MapReduce, SQL DBs) have their own tools

The basic unit is Directed Acyclic Graph (DAG)

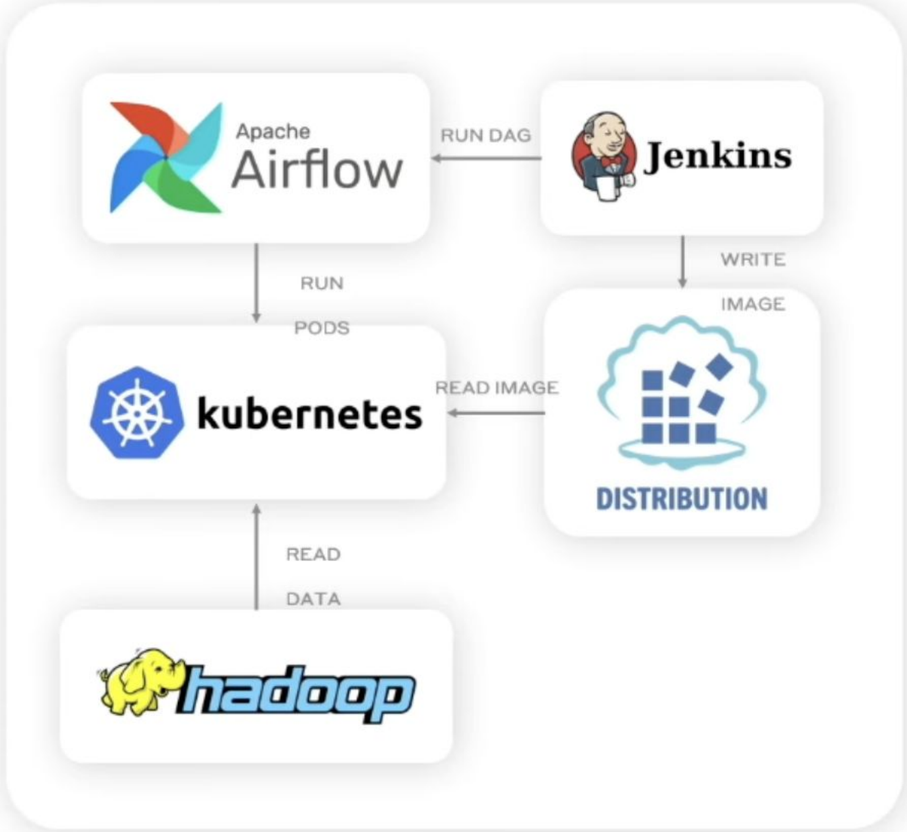


Apache Airflow

Example of autoML system

https://www.youtube.com/watch?v=HgrF4Ta2EY0

01 Где запускать код?	• Kubernetes • Registry
02 Откуда брать данные?	• Hadoop
03 Как оркестрировать запуски?	• Airflow
04 Как собирать образы?	• Jenkins
05 Где хранить артефакты?	

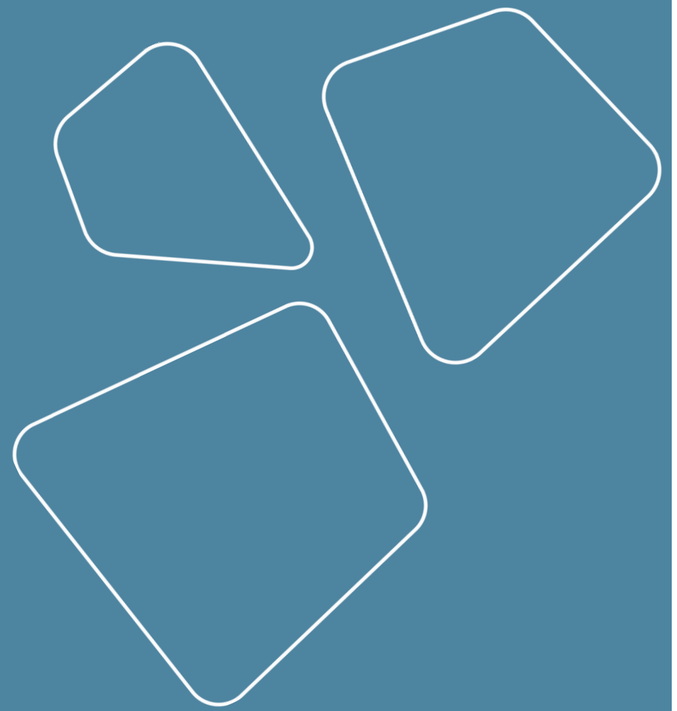


Airflow

girafe
ai

04

Timeline



- Airflow overview
- Architecture components
- DAG
- Control Flow
- Operators
- Task States
- UI

Workflows as code

- **Dynamic**: Airflow pipelines are configured as Python code, allowing for dynamic pipeline generation.
- **Extensible**: The Airflow framework contains operators to connect with numerous technologies. All Airflow components are extensible to easily adjust to your environment.
- **Flexible**: Workflow parameterization is built-in leveraging the Jinja templating engine.

Example: DAG

```python
from datetime import datetime

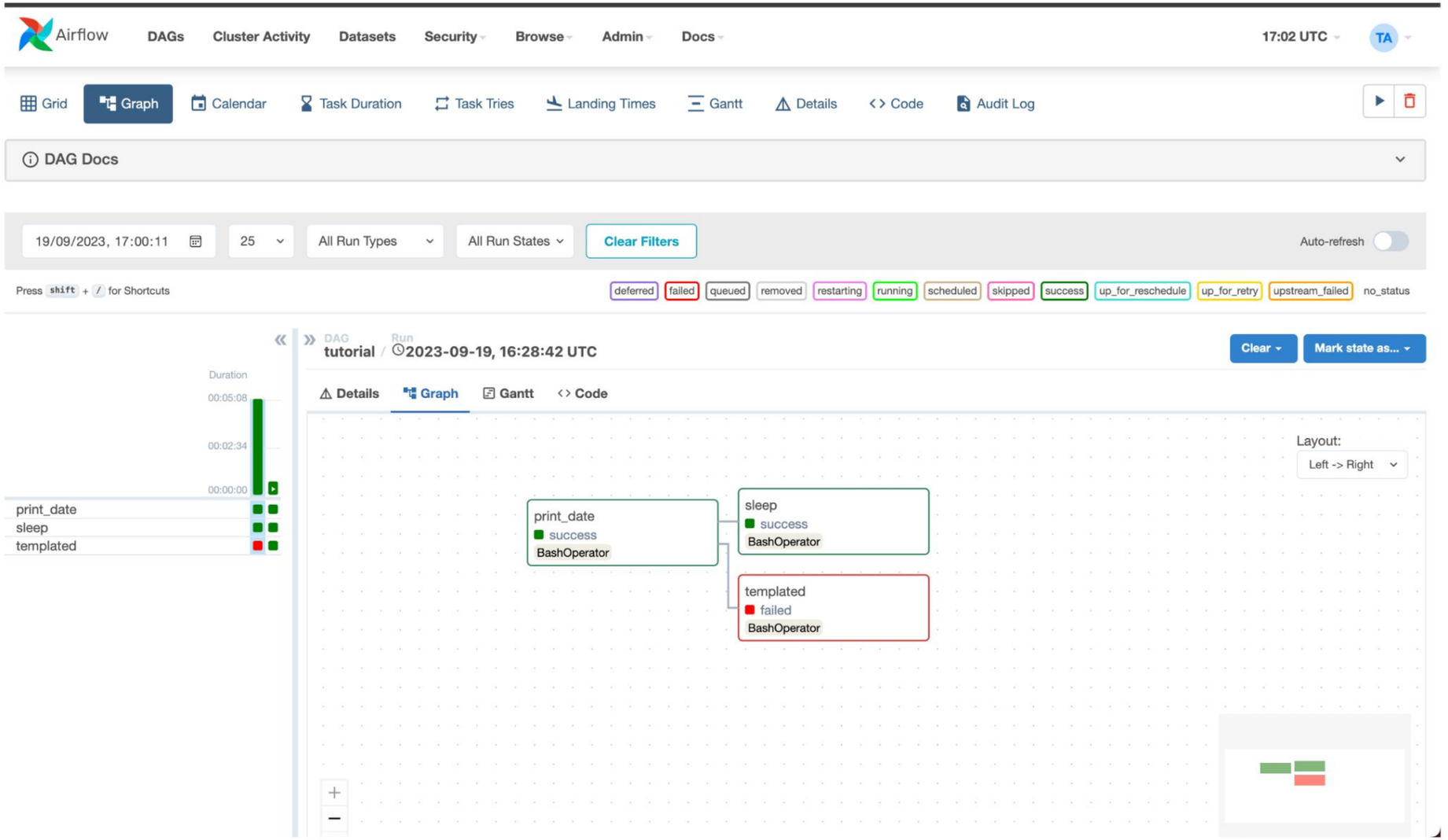
from airflow import DAG
from airflow.decorators import task
from airflow.operators.bash import BashOperator

# A DAG represents a workflow, a collection of tasks
with DAG(dag_id="demo", start_date=datetime(2022, 1, 1), schedule="0 0 * * *") as dag:
    # Tasks are represented as operators
    hello = BashOperator(task_id="hello", bash_command="echo hello")

    @task()
    def airflow():
        print("airflow")

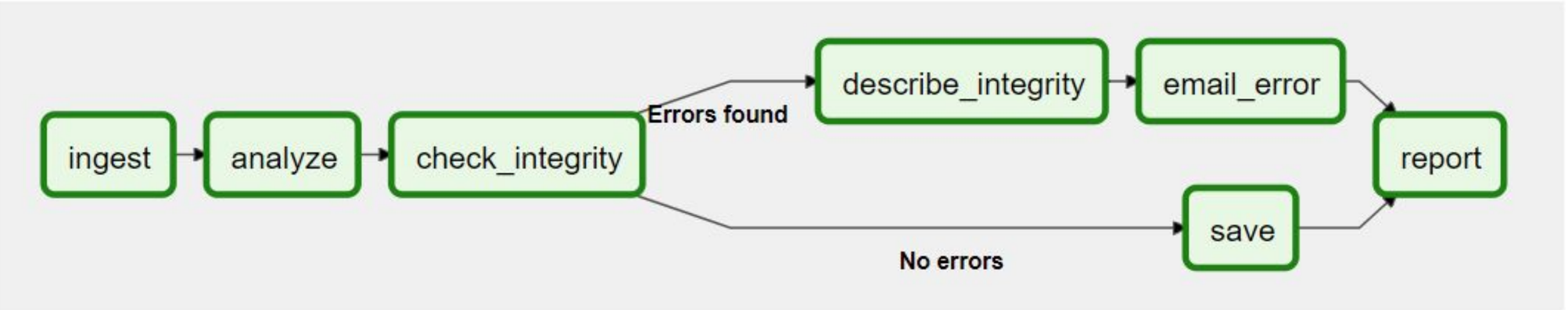
    # Set dependencies between tasks
    hello >> airflow()
```

Example: UI


Airflow
DAGs
Cluster Activity
Datasets
Security
Browse
Admin
Docs
17:02 UTC
TA
Grid
Graph
Calendar
Task Duration
Task Tries
Landing Times
Gantt
Details
Code
Audit Log
DAG Docs
19/09/2023, 17:00:11
25
All Run Types
All Run States
Clear Filters
Auto-refresh
Press shift + / for Shortcuts
deferred
failed
queued
removed
restarting
running
scheduled
skipped
success
up_for_reschedule
up_for_retry
upstream_failed
no_status
DAG
tutorial
Run
2023-09-19, 16:28:42 UTC
Clear
Mark state as...
Duration
00:05:08
00:02:34
00:00:00
print_date
sleep
templated
Details
Graph
Gantt
Code
Layout:
Left -> Right
print_date
success
BashOperator
sleep
success
BashOperator
templated
failed
BashOperator

Why Airflow?

- Workflows can be stored in *version control* so that you can roll back to previous versions
- Workflows can be *developed by multiple people simultaneously*
- *Tests can be written* to validate functionality
- Components are extensible and *you can build on a wide collection of existing components*



Why not Airflow?

- Бесконечно работающие процессы
- Основной метод управления - написание кода на python. То есть не drag-n-drop в ui
- Не подходит для компаний с десятками тысяч процессов, так как планировщик не справится

Architecture components (Base)

- **Scheduler**. Управляет как запуском запланированных рабочих процессов, так и отправкой задачи на исполнение конкретному воркеру (исполнителю).

- **Web-server**, который предоставляет удобный пользовательский интерфейс для инспекции, запуска и отладки поведения DAG и задач.

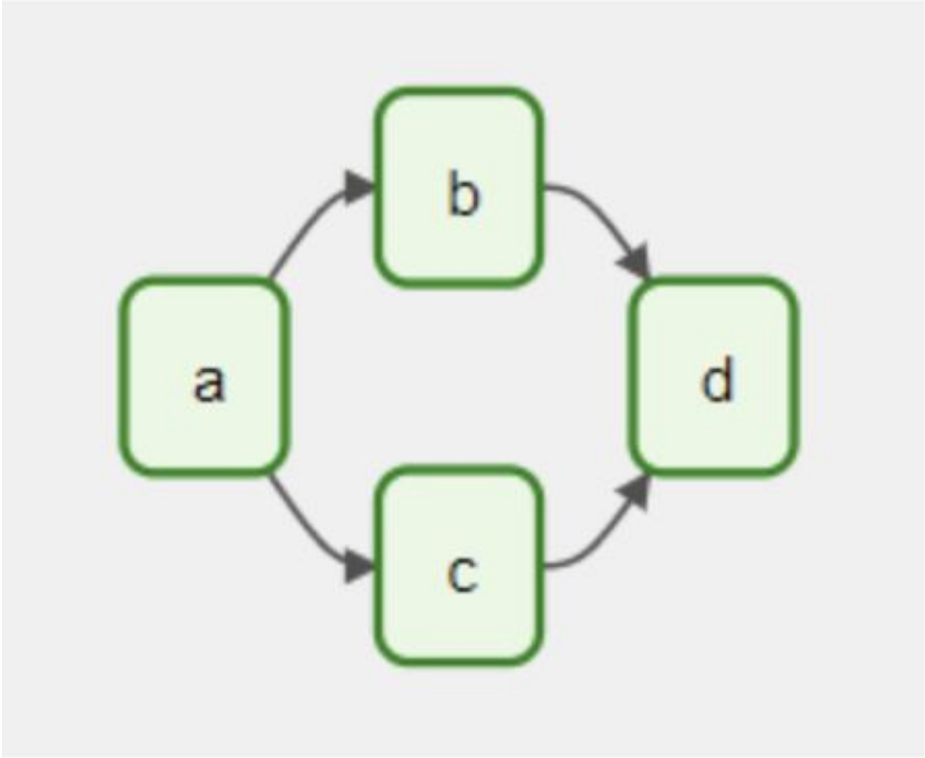
- **Директория с python-файлами DAG**, которую читает планировщик, чтобы определить, какие задачи выполнять и когда их выполнять.

- **База данных метаданных**, которую компоненты Airflow используют для хранения состояния рабочих процессов и задач.

Architecture components (Optional)

- **Optional Worker**: исполнитель, который выполняет задачи, переданные ему планировщиком. Если в базовой конфигурации исполнитель является частью планировщика, то опционально его можно сконфигурировать как процесс в Celery или как под в K8s
- **Optional triggerer**. Позволяет выполнять задачи при наступлении определенного события
- **Optional DAG processor**. Полезен, если нужно настроить чтение и синхронизацию с базой данных метаданных не через процесс шедулера, а как отдельную компоненту.
- **Optional folder of plugins**. Способ расширить функциональность операторов. Плагины читаются планировщиком, процессором дагов, триггером и веб-сервером.

DAG - Directed Acyclic Graph



Declaring a DAG

via context manager

```
import datetime

from airflow import DAG
from airflow.operators.empty import EmptyOperator

with DAG(
    dag_id="my_dag_name",
    start_date=datetime.datetime(2021, 1, 1),
    schedule="@daily",
):
    EmptyOperator(task_id="task")
```

Declaring a DAG

via standard constructor

```
import datetime

from airflow import DAG
from airflow.operators.empty import EmptyOperator

my_dag = DAG(
    dag_id="my_dag_name",
    start_date=datetime.datetime(2021, 1, 1),
    schedule="@daily",
)
EmptyOperator(task_id="task", dag=my_dag)
```

Declaring a DAG

via @dag wrapper

```
import datetime

from airflow.decorators import dag
from airflow.operators.empty import EmptyOperator


@dag(start_date=datetime.datetime(2021, 1, 1), schedule="@daily")
def generate_dag():
    EmptyOperator(task_id="task")


generate_dag()
```

Task Dependencies

```
first_task >> [second_task, third_task]
third_task << fourth_task
```

Running DAGs

Процесс будет запущен в одном из трех случаев:

- Вручную через API
- Если сработал триггер
- По расписанию, определенному при создании графа

```
with DAG("my_daily_dag", schedule="@daily"):
    ...
```

```
with DAG("my_daily_dag", schedule="0 0 * * *"):
    ...

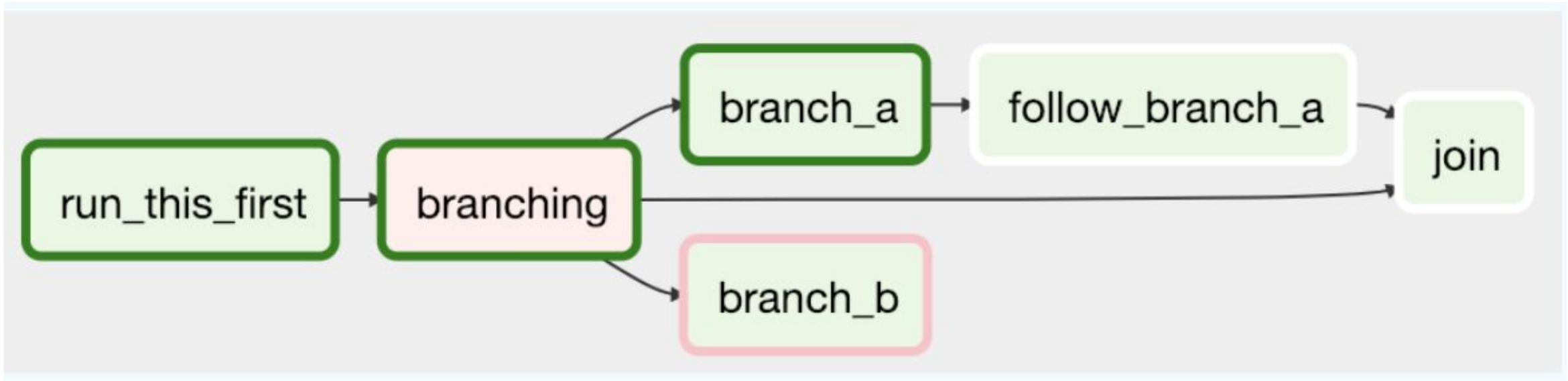
with DAG("my_one_time_dag", schedule="@once"):
    ...

with DAG("my_continuous_dag", schedule="@continuous"):
    ...
```

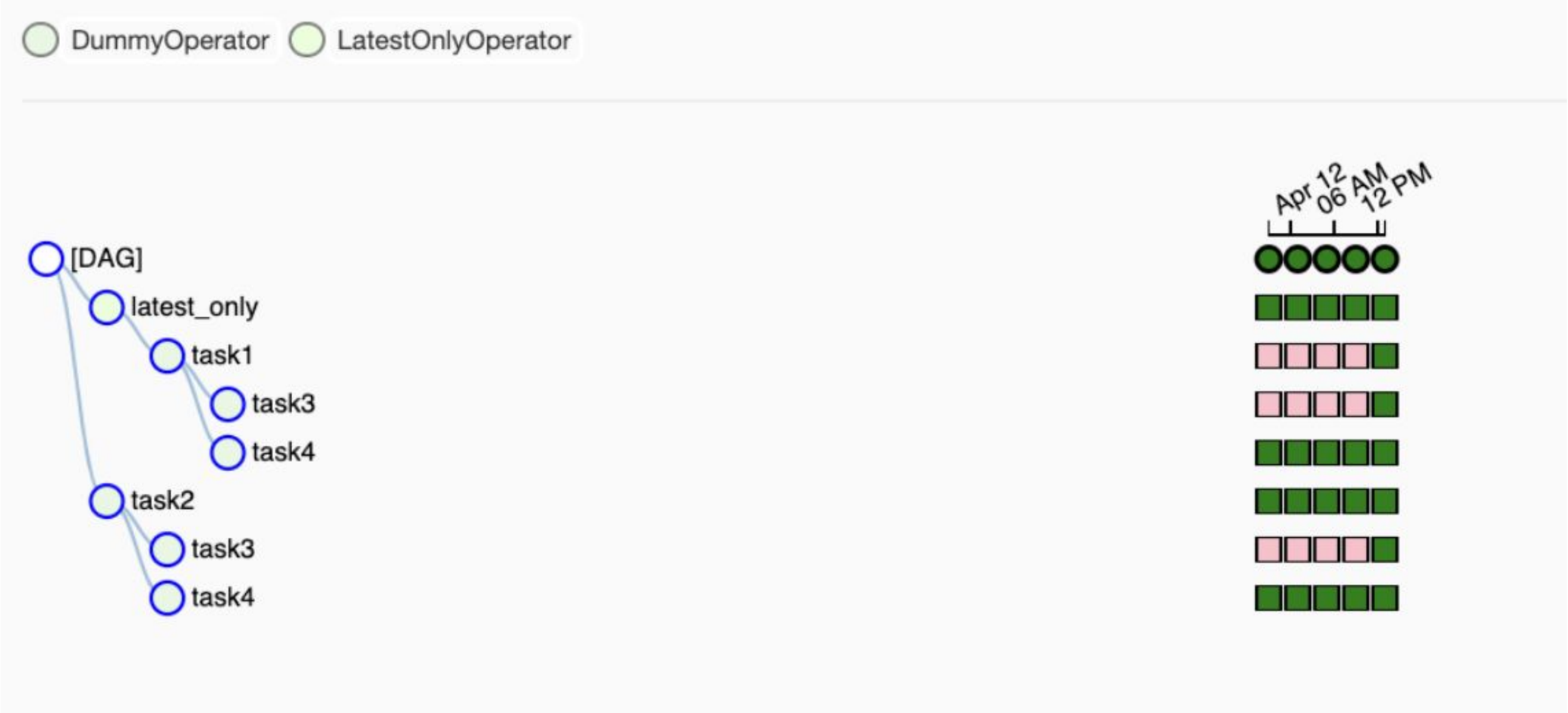
Control Flow

- **Branching** - выбор Задачи для перехода на основании некоторого условия
- **Trigger Rules** - установка условий, при которых ДАГ будет выполнять задачу
- **Setup and Tear Down** - определение отношений между setup и tear-down
- **Latest Only** - особая форма ветвления, которая работает только с ДАГ, выполняющимися для текущего момента
- **Depends On Past** - задачи могут зависеть от своего предыдущего выполнения

Control Flow: Branching



Control Flow: Latest Only

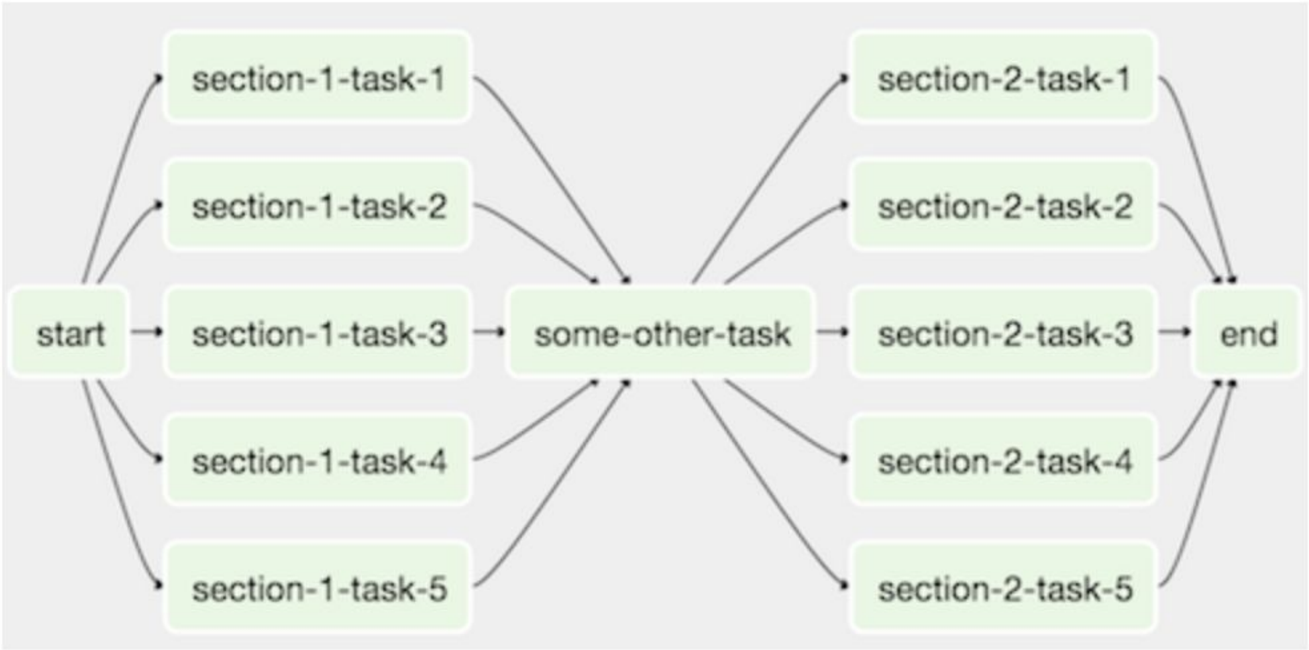


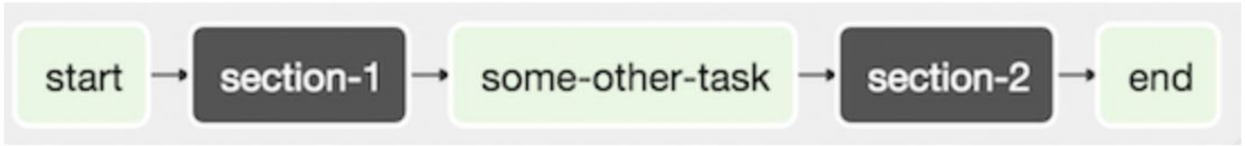
Control Flow: Depends on Past

Control Flow: Trigger rules

- all_success (по умолчанию): Все задачи выше по потоку выполнены успешно
- all_failed: Все задачи выше по потоку находятся в состоянии failed
- all_done: Все задачи выше по потоку завершили выполнение
- all_skipped: Все задачи выше по потоку в состоянии пропуска
- one_failed: Хотя бы одна задача выше по потоку в состоянии failed
- one_success: Хотя бы одна задача выше по потоку выполнена успешно
- one_done: Хотя бы одна задача выше по потоку выполнена успешно
- none_failed: Все задачи выше по потоку выполнены успешно или пропущены
- none_skipped: Ни одна из задач выше по потоку не находится в состоянии skipped - то есть, все задачи выше по потоку находятся в состоянии success, failed или upstream_failed
- always: Нет зависимостей вообще, выполнить эту задачу в любое время

SubDAG


start
section-1-task-1
section-1-task-2
section-1-task-3
section-1-task-4
section-1-task-5
some-other-task
section-2-task-1
section-2-task-2
section-2-task-3
section-2-task-4
section-2-task-5
end


start
section-1
some-other-task
section-2
end

Operators

- BashOperator
- PythonOperator
- EmailOperator
- HttpOperator
- MySqlOperator
- PostgresOperator
- MsSqlOperator
- OracleOperator
- JdbcOperator
- DockerOperator
- HiveOperator
- S3FileTransformOperator
- PrestoToMySqlOperator
- SlackAPIOperator

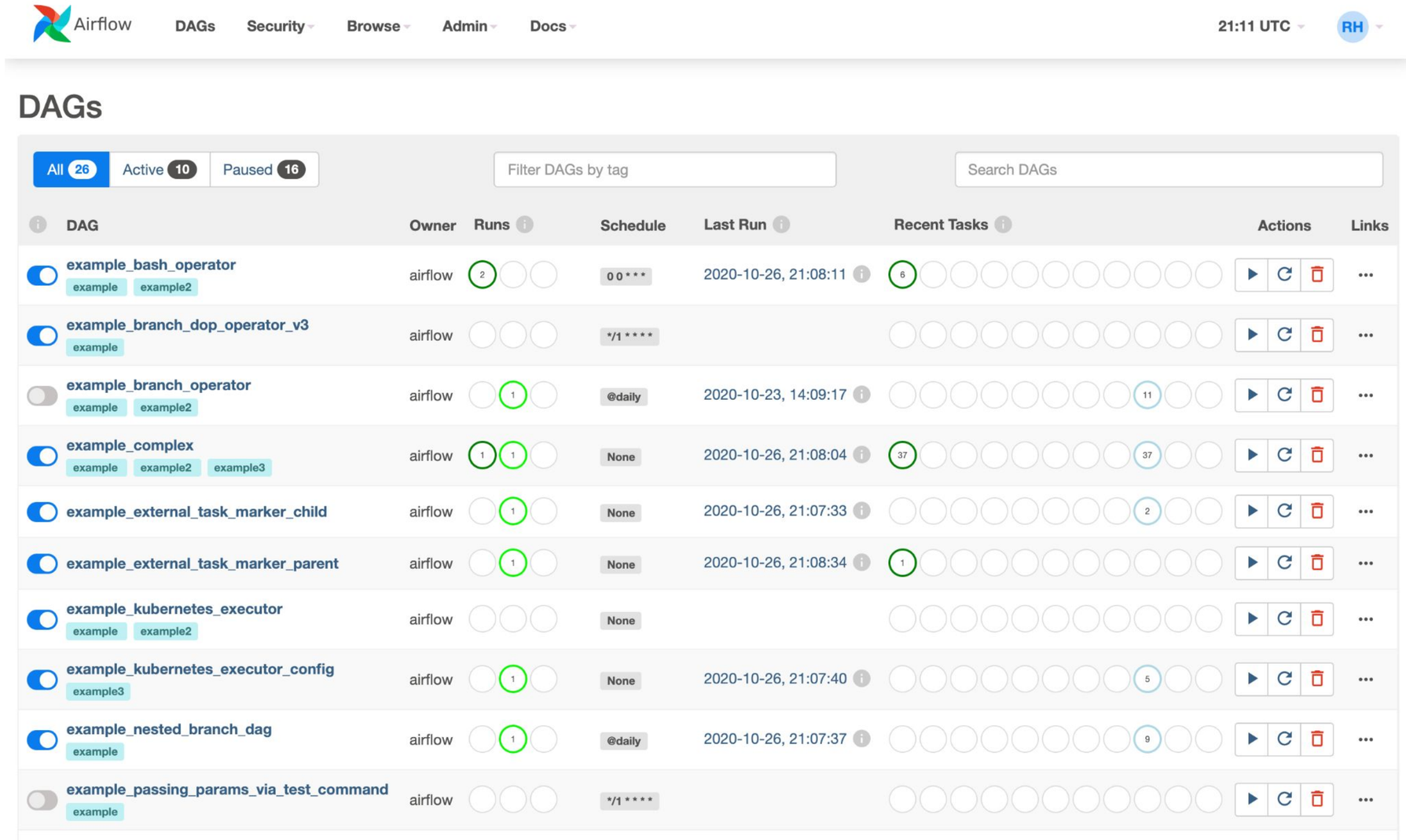
Task States

- **none**: Задача еще не была поставлена в очередь на выполнение.
- **scheduled**: Планировщик определил, что зависимости задачи выполнены и она должна быть запущена.
- **queued**: Задача была назначена исполнителю (Executor) и ожидает в очереди воркера.
- **running**: Задача выполняется воркером.
- **success**: Задача завершила выполнение без ошибок.
- **restarting**: Задача была перзапущена.
- **failed**: Во время выполнения задачи произошла ошибка.
- **skipped**: Задача была пропущена (из-за ветвления, LatestOnly или похожих причин).
- **upstream_failed**: Предшествующая задача не была выполнена.
- **up_for_retry**: Задача не была выполнена, но у неё остались попытки на повторное выполнение и она будет запланирована заново.
- **deferred**: Задача была отложена до срабатывания триггера.

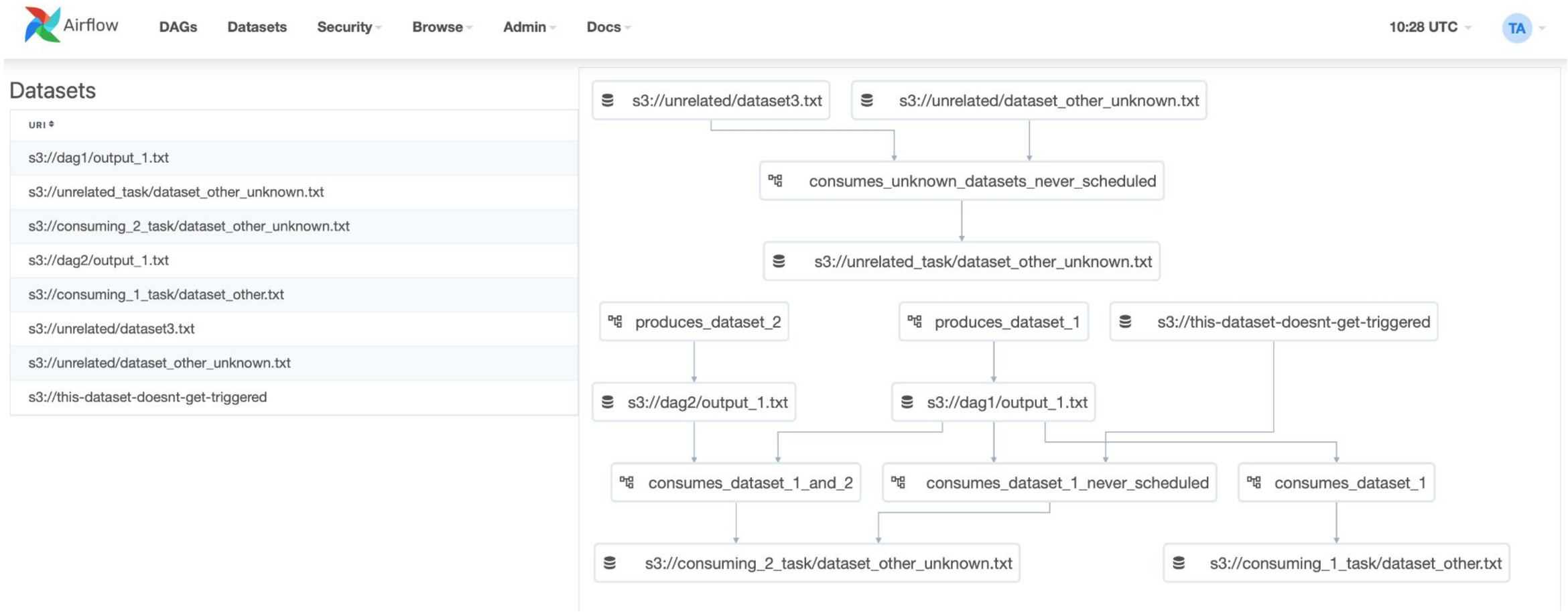
Zombie Task States

- Задачи-зомби (**Zombie Tasks**) — это экземпляры задач, “застрявшие” в состоянии выполнения, несмотря на то, что связанные с ними задания неактивны. Airflow периодически обнаруживает их, очищает и, в зависимости от настроек, либо отмечает задачу как неудачную, либо пытается повторно ее запустить.
- Неумирающие задачи (**Undead tasks**) — это задачи, которые не должны выполняться, но выполняются. Airflow периодически находит их и завершает выполнение.

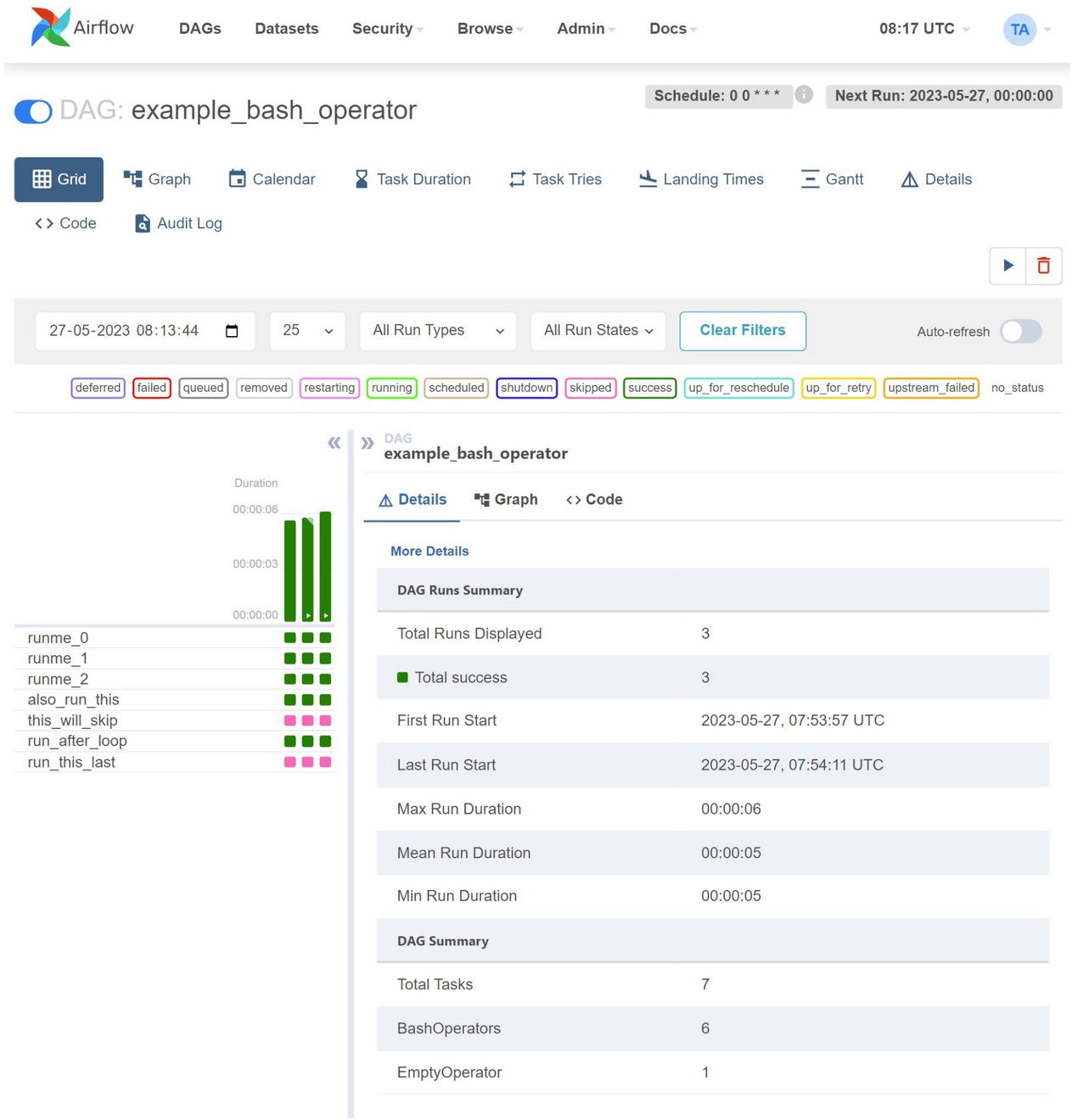
UI: DAGs View


Airflow
DAGs
Security
Browse
Admin
Docs
21:11 UTC
RH
DAGs
All 26
Active 10
Paused 16
Filter DAGs by tag
Search DAGs
DAG
Owner
Runs
Schedule
Last Run
Recent Tasks
Actions
Links
example_bash_operator
example
example2
airflow
0 0 * * *
2020-10-26, 21:08:11
example_branch_dop_operator_v3
example
airflow
*/1 * * * *
example_branch_operator
example
example2
airflow
@daily
2020-10-23, 14:09:17
example_complex
example
example2
example3
airflow
None
2020-10-26, 21:08:04
example_external_task_marker_child
airflow
None
2020-10-26, 21:07:33
example_external_task_marker_parent
airflow
None
2020-10-26, 21:08:34
example_kubernetes_executor
example
example2
airflow
None
example_kubernetes_executor_config
example3
airflow
None
2020-10-26, 21:07:40
example_nested_branch_dag
example
airflow
@daily
2020-10-26, 21:07:37
example_passing_params_via_test_command
example
airflow
*/1 * * * *

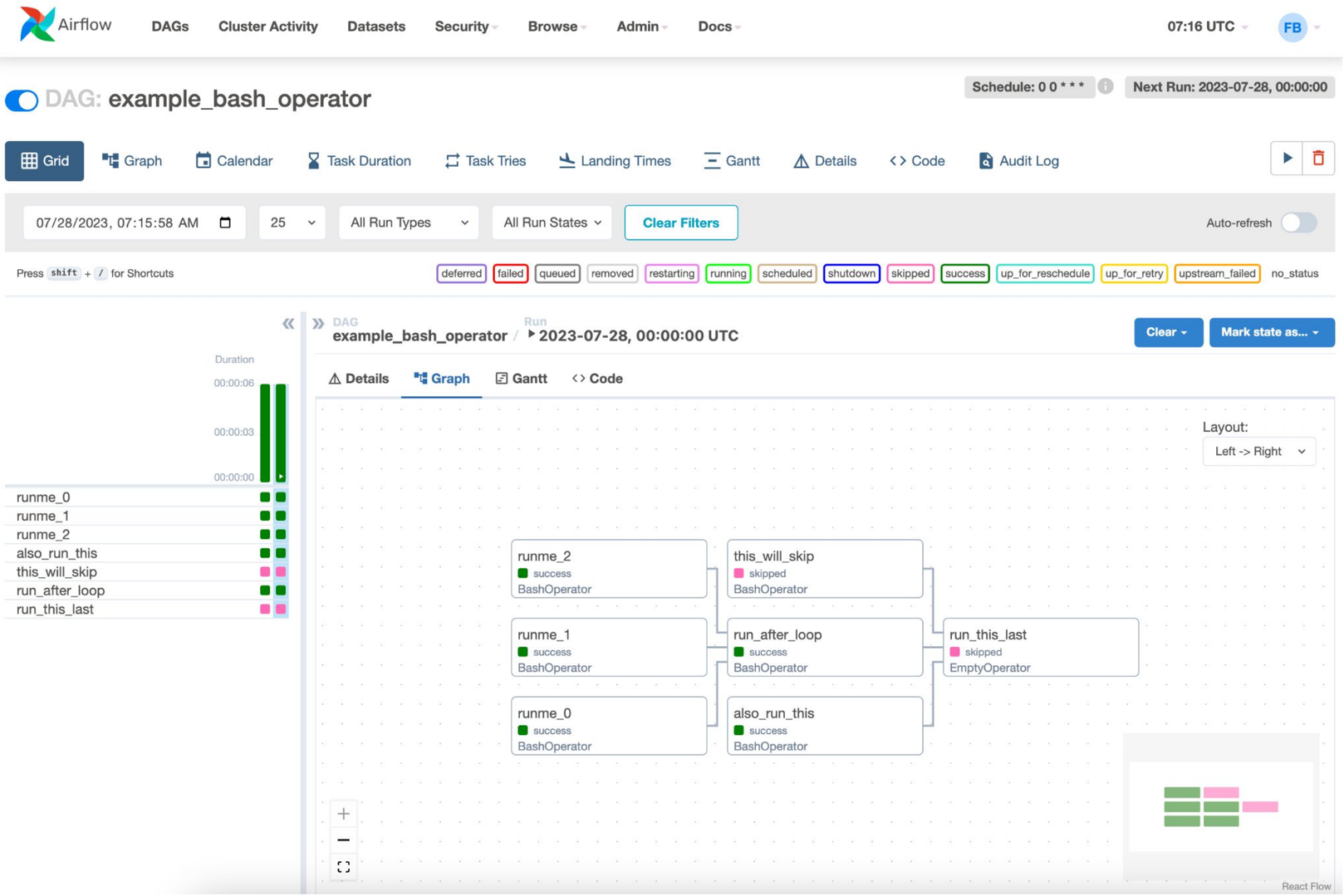
UI: Datasets View



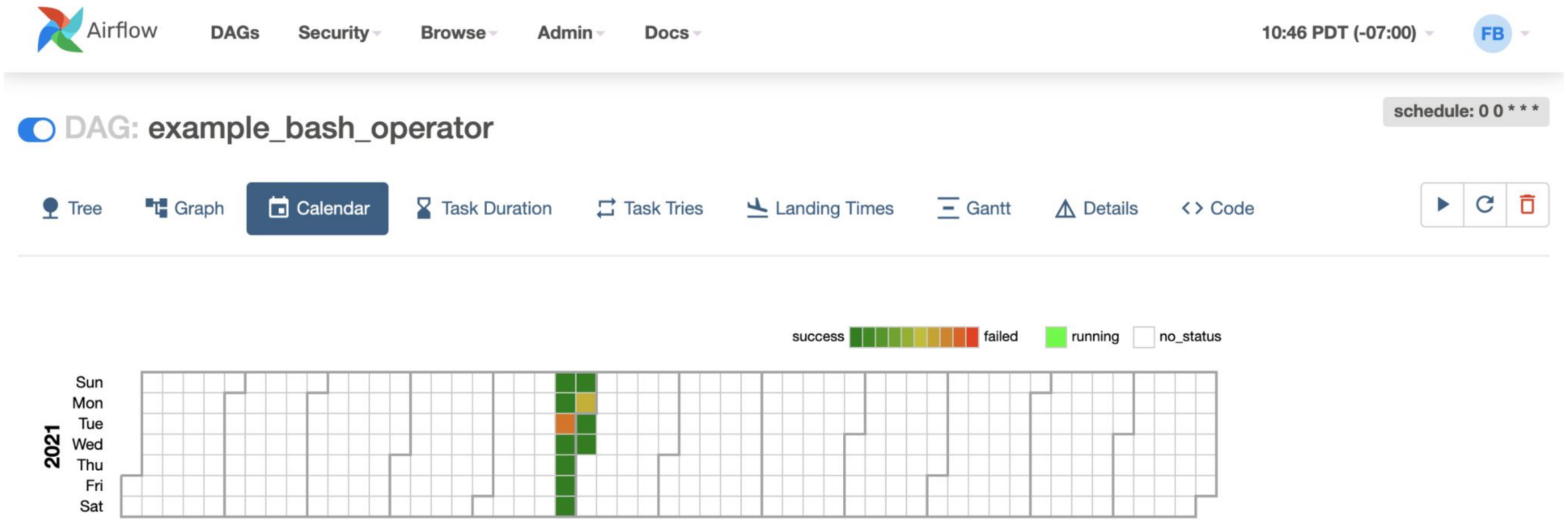
UI: Grid View


Airflow
DAGs
Datasets
Security
Browse
Admin
Docs
08:17 UTC
TA
DAG: example_bash_operator
Schedule: 0 0 * * *
Next Run: 2023-05-27, 00:00:00
Grid
Graph
Calendar
Task Duration
Task Tries
Landing Times
Gantt
Details
Code
Audit Log
27-05-2023 08:13:44
25
All Run Types
All Run States
Clear Filters
Auto-refresh
deferred
failed
queued
removed
restarting
running
scheduled
shutdown
skipped
success
up_for_reschedule
up_for_retry
upstream_failed
no_status
Duration
00:00:06
00:00:03
00:00:00
runme_0
runme_1
runme_2
also_run_this
this_will_skip
run_after_loop
run_this_last
DAG
example_bash_operator
Details
Graph
Code
More Details
DAG Runs Summary
Total Runs Displayed 3
Total success 3
First Run Start 2023-05-27, 07:53:57 UTC
Last Run Start 2023-05-27, 07:54:11 UTC
Max Run Duration 00:00:06
Mean Run Duration 00:00:05
Min Run Duration 00:00:05
DAG Summary
Total Tasks 7
BashOperators 6
EmptyOperator 1

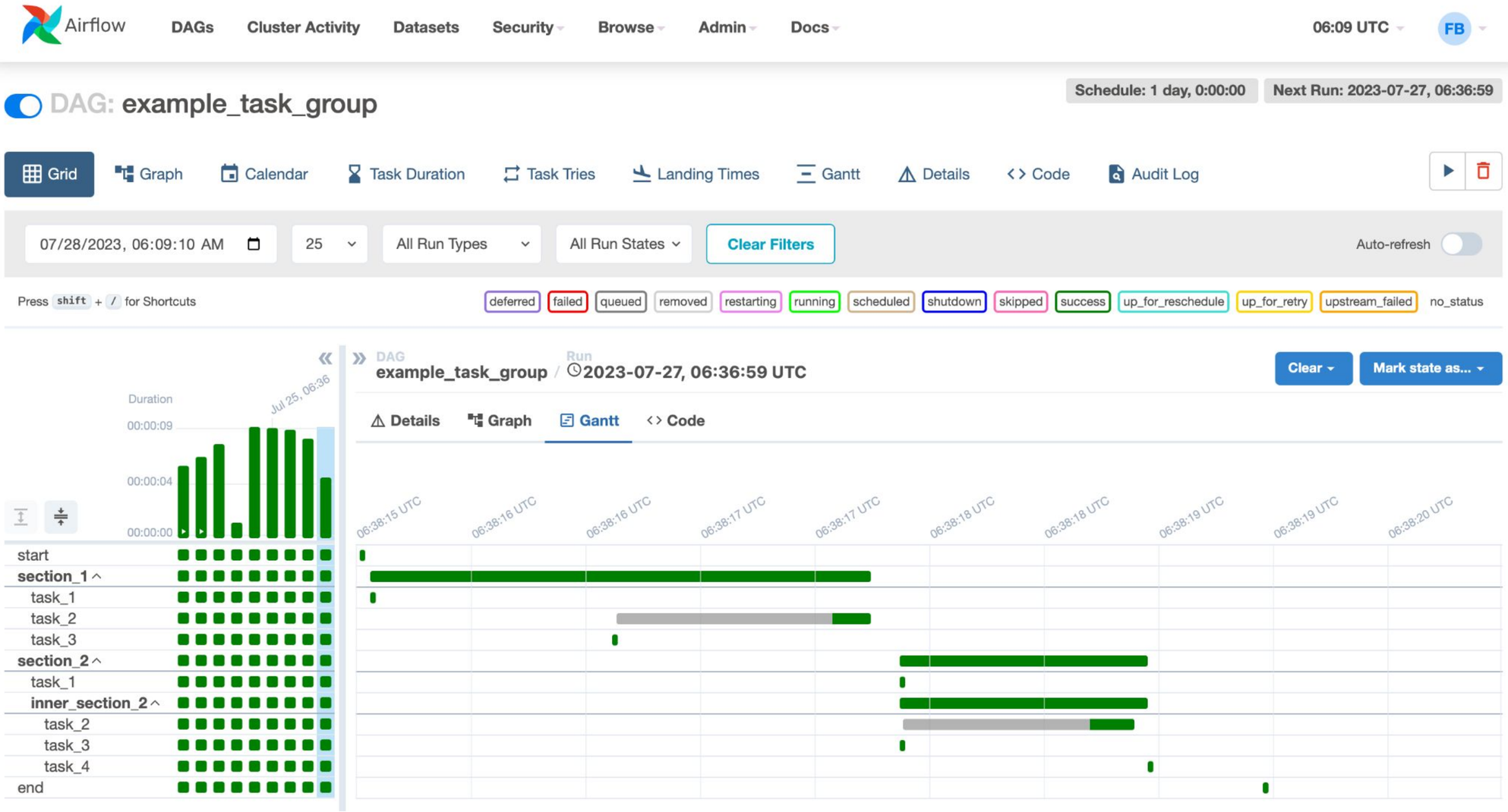
UI: Graph View



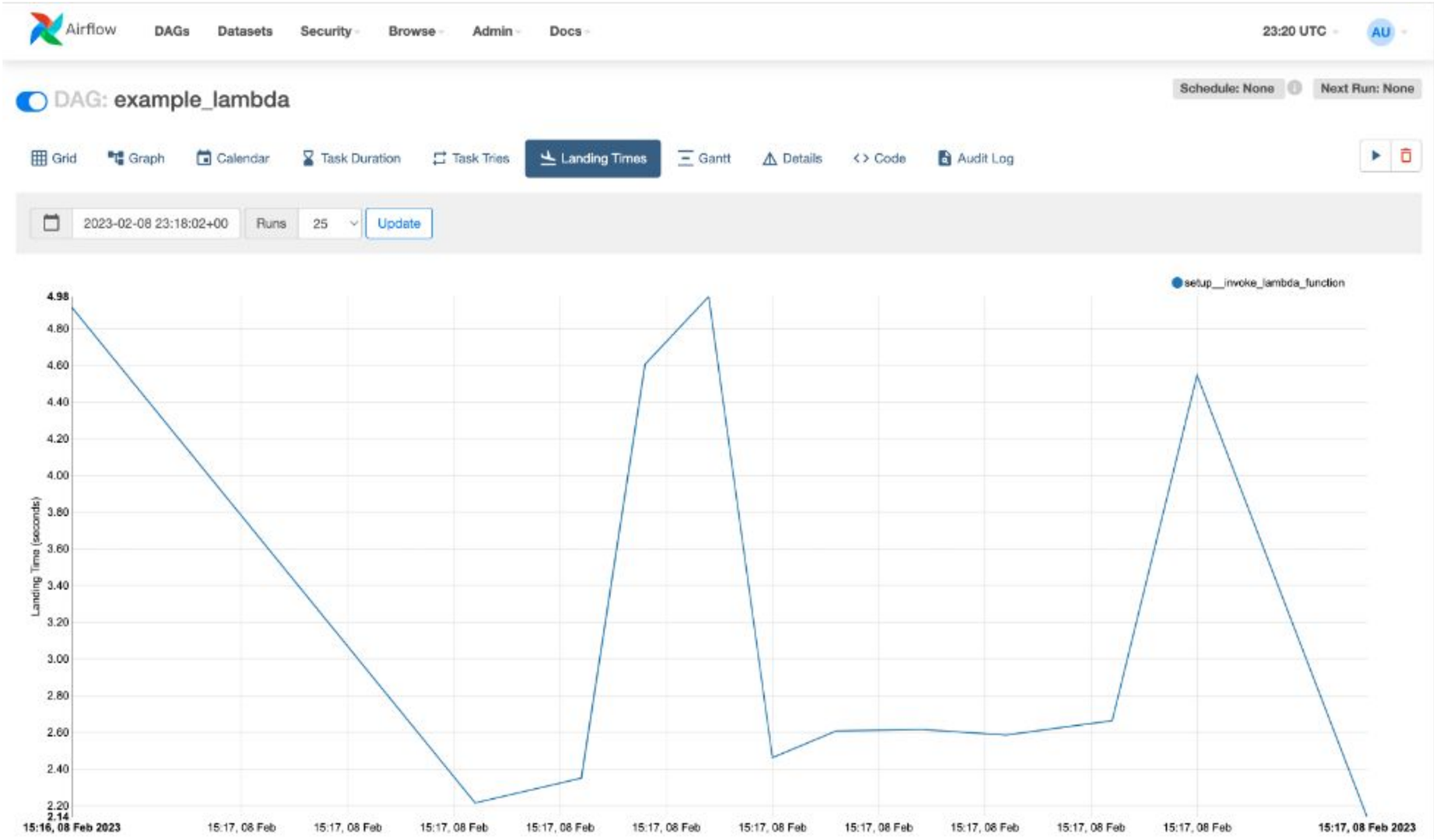
UI: Calendar View



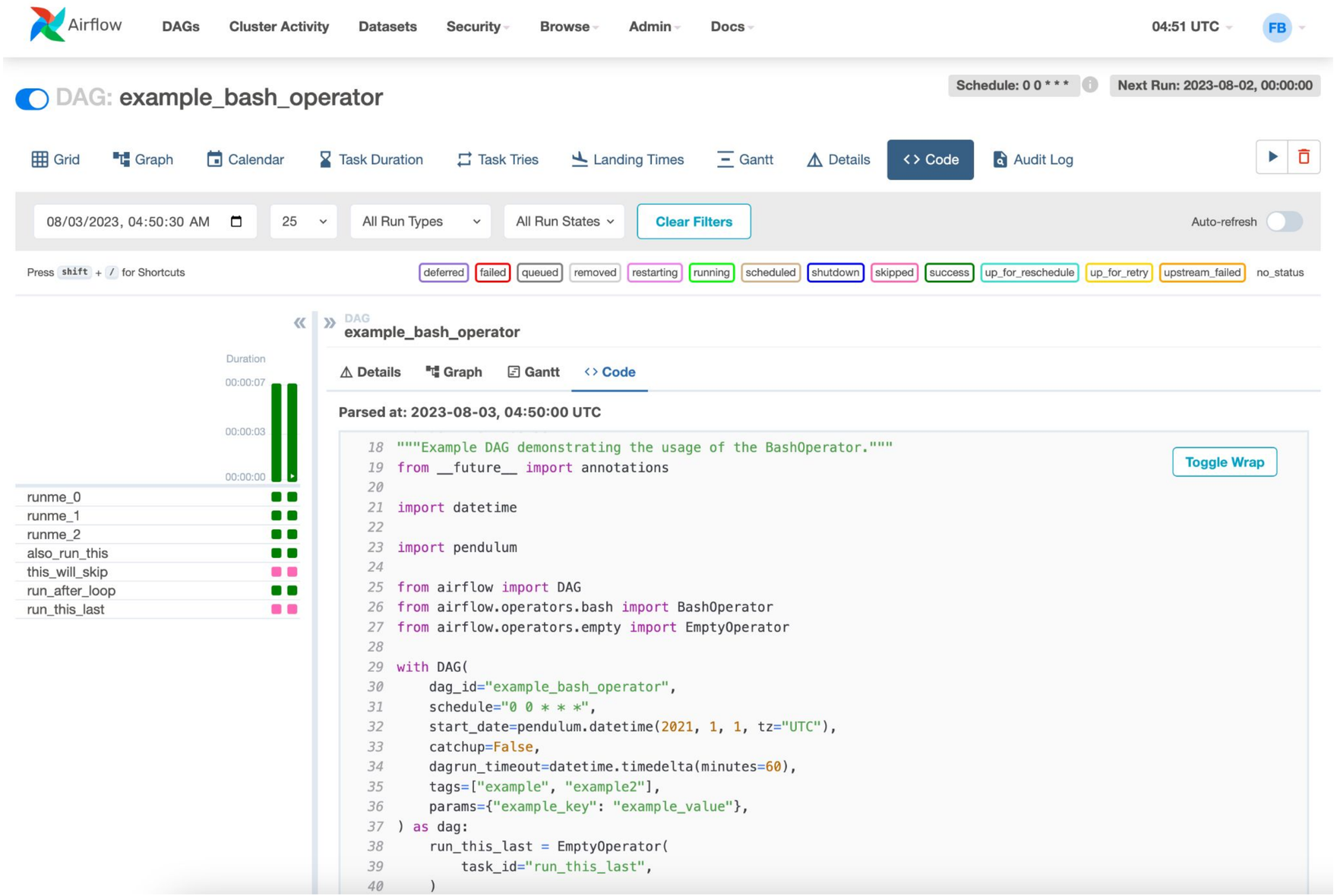
UI: Gantt Chart View



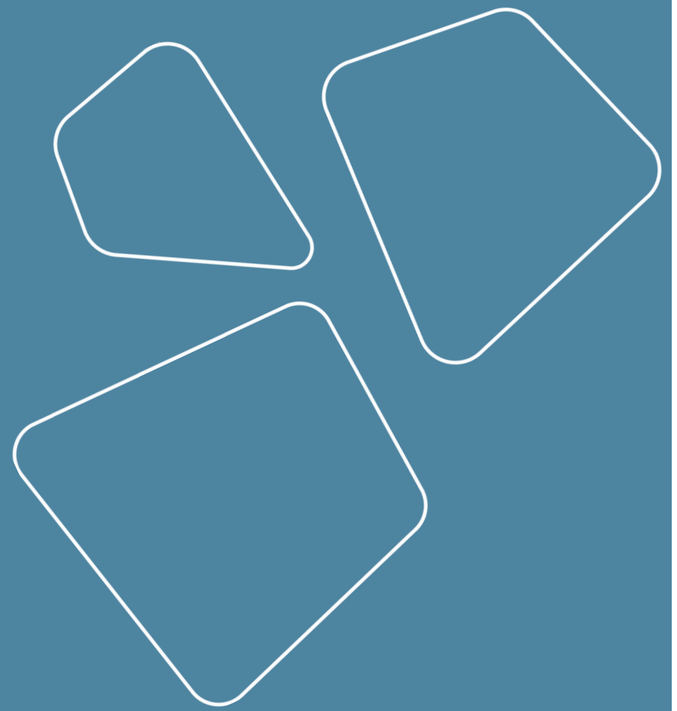
UI: Landing Times View



UI: Code View


Airflow
DAGs
Cluster Activity
Datasets
Security
Browse
Admin
Docs
04:51 UTC
FB
DAG: example_bash_operator
Schedule: 0 0 * * *
Next Run: 2023-08-02, 00:00:00
Grid
Graph
Calendar
Task Duration
Task Tries
Landing Times
Gantt
Details
Code
Audit Log
08/03/2023, 04:50:30 AM
25
All Run Types
All Run States
Clear Filters
Auto-refresh
Press shift + / for Shortcuts
deferred
failed
queued
removed
restarting
running
scheduled
shutdown
skipped
success
up_for_reschedule
up_for_retry
upstream_failed
no_status
Duration
00:00:07
00:00:03
00:00:00
runme_0
runme_1
runme_2
also_run_this
this_will_skip
run_after_loop
run_this_last
DAG
example_bash_operator
Details
Graph
Gantt
Code
Parsed at: 2023-08-03, 04:50:00 UTC
Toggle Wrap
18 """Example DAG demonstrating the usage of the BashOperator."""
19 from __future__ import annotations
20
21 import datetime
22
23 import pendulum
24
25 from airflow import DAG
26 from airflow.operators.bash import BashOperator
27 from airflow.operators.empty import EmptyOperator
28
29 with DAG(
30 dag_id="example_bash_operator",
31 schedule="0 0 * * *",
32 start_date=pendulum.datetime(2021, 1, 1, tz="UTC"),
33 catchup=False,
34 dagrun_timeout=datetime.timedelta(minutes=60),
35 tags=["example", "example2"],
36 params={"example_key": "example_value"},
37) as dag:
38 run_this_last = EmptyOperator(
39 task_id="run_this_last",
40)

Recap



- Contours of computation
  - Offline
  - Online (realtime)
  - Near real time
  - Edge
- Types of compute resources
- Jobs runners

Thanks for attention!

Questions?

